Cost of Living Analytics – AI-Powered Forecast Platform Documentation

1. INTRODUCTION

In an era marked by heightened economic volatility, the ability to understand and, more importantly, anticipate changes in the cost of living has become a critical necessity. This volatility affects every layer of society—individuals and families managing household budgets, businesses shaping pricing strategies and supply chains, and policymakers tasked with guiding national economic stability. The “Cost of Living Analytics” platform stands as an advanced, AI-driven web application designed to decode these complex economic dynamics. Its core mission is to deliver accessible, data-driven forecasts for India’s Consumer Price Index (CPI), with a particular focus on the General Index. At its foundation, the platform transforms static historical data into actionable insights. By ingesting extensive economic records and channeling them through a sophisticated analytical pipeline, it produces a coherent and forward-looking economic narrative. Central to this process is a Deep Learning Neural Network model built using Keras and TensorFlow. Trained meticulously on historical CPI data, the model captures intricate, non-linear relationships and subtle temporal patterns—including seasonal variations—that define economic behavior. It not only learns the “what” of price movements but also uncovers the “how” and “when” behind their fluctuations.

The predictive intelligence of this model is presented through an intuitive, interactive dashboard developed with Streamlit and Plotly. The design emphasizes clarity and immediacy: users can simply select a future year and month to generate on-demand CPI forecasts. Results are visualized through elegant gauge charts, offering clear and immediate interpretation. To enhance contextual understanding, historical data is displayed as a solid continuous line that seamlessly transitions into a dotted line representing model predictions—creating an unbroken narrative that connects past, present, and future trends.

Ultimately, this platform empowers a diverse audience—financial analysts, researchers, policymakers, students, and curious individuals alike—to make informed, data-driven decisions. By bridging the gap between raw economic data and strategic foresight, the Cost of Living Analytics platform transforms complexity into clarity and prediction into empowerment.

2. OBJECTIVES

The primary objective of this project is to design, develop, and deploy a comprehensive end-to-end system for forecasting the Cost of Living Index, thereby offering a clear predictive perspective on economic volatility. To achieve this overarching goal, a series of structured and interdependent objectives were defined.

The process begins with the collection and consolidation of historical Consumer Price Index (CPI) data for India, with a deliberate emphasis on maintaining the distinction between the 'Rural' and 'Urban' sector datasets. Following this, the project advances to an extensive data preprocessing and feature engineering phase—one that transcends mere data cleaning. This stage involves addressing missing values to preserve data integrity, numerically encoding temporal attributes such as month names, and applying one-hot encoding to categorical variables like *Sector*, thus producing a refined, model-ready dataset enriched with meaningful features.

Next, the focus shifts to developing and training a Deep Learning regression model built using Keras, selected for its capacity to learn intricate, non-linear patterns often overlooked by traditional statistical models. The model is trained on 80% of the dataset, while its generalization performance is evaluated on the remaining 20% test data using Mean Squared Error (MSE) and Mean Absolute Error (MAE) as key evaluation metrics.

A vital architectural milestone involves persisting the best-performing model alongside its associated StandardScaler, thereby establishing a seamless bridge between the offline training environment and the deployed, real-time application.

Finally, these technical foundations converge in the creation of an interactive Streamlit web dashboard. This component "productizes" the predictive model, allowing users to generate on-demand CPI forecasts through an intuitive interface. Visual outputs are rendered via Plotly-based gauge and trend charts, offering immediate, interpretable insights. The dashboard's design emphasizes responsiveness, accessibility, and user experience, ensuring that advanced economic forecasting is delivered in a modern and approachable format.

3. TOOLS & DOCUMENTS

This project is built upon a carefully curated, modern, and fully open-source technology stack that integrates data science, machine learning, and web development tools. All dependencies and their exact versions are comprehensively documented in the `requirements.txt` file to ensure reproducibility and environment consistency.

The core programming language used throughout the project—from initial data exploration to final deployment—is Python 3.11+, chosen for its versatility and rich ecosystem of scientific and web development libraries.

For the machine learning and data science components, the project is anchored in the TensorFlow (v2.20.0) ecosystem, which provides the computational backbone for

numerical processing and model training. Building on this foundation, the Keras API (v3.11.3) was employed for the intuitive design, configuration, and training of the deep neural network architecture, enabling a streamlined and efficient model development process.

Category	Library/ Tool	Version	Purpose in Project
Core Language	Python	3.11+	The foundational language for all scripts and libraries.
Machine Learning	TensorFlow	2.20.0	The core backend for building and executing the neural network.
	Keras	3.11.3	High-level API used to define the sequential model architecture.
Data Science	Scikit-learn	1.7.2	Used for `train_test_split` and `StandardScaler` for data preprocessing.
	Pandas	2.3.3	Primary tool for data loading, cleaning, and manipulation (DataFrames).
	NumPy	2.3.4	Used for high-performance numerical operations and array transformations.
Web Application	Streamlit	1.50.0	The core framework for building the interactive web dashboard.
Data Visualization	Plotly	6.3.1	Used to create all interactive gauge charts and line plots.

The machine learning pipeline is reinforced by several essential preprocessing and data manipulation libraries. Scikit-learn (v1.7.2) played a pivotal role in key preprocessing tasks, particularly for data partitioning using `train_test_split` and feature normalization via `StandardScaler`. Pandas (v2.3.3) served as the backbone for data ingestion, cleaning, transformation, and manipulation, while NumPy (v2.3.4) provided optimized array structures and high-performance numerical operations essential for handling scaled data efficiently.

For the web application and visualization layer, Streamlit (v1.50.0) was selected as the primary framework due to its simplicity, Python-native integration, and rapid development capabilities. Streamlit facilitated the seamless conversion of the trained model into an interactive, user-friendly, and responsive web dashboard. Complementing this, Plotly (v6.3.1) was employed to craft sophisticated, interactive

visualizations—including animated gauge charts and historical-versus-predicted trend lines—that enhance interpretability and user engagement.

In addition to these core libraries, standard Python modules such as `calendar` (for month-name-to-number mapping) and `os` (for file path and environment management) were utilized for various utility functions. The initial experimentation, data exploration, and model prototyping were conducted in a Jupyter Notebook environment, providing an ideal platform for iterative development, visualization, and performance tuning prior to deployment.

4. SYSTEM ARCHITECTURE

The system is logically divided into two main components: the Offline Training Pipeline for model creation and the Online Inference Application for serving real-time user predictions.

Offline Training Pipeline (infosys-internship-project-1.ipynb):

This phase, implemented within a Jupyter Notebook, is dedicated to developing and training the predictive deep learning model. The process begins with data ingestion, where the file *All_India_Index_july2019_20Aug2020.csv* is imported into a Pandas DataFrame. The dataset then undergoes comprehensive data cleaning, which includes handling missing (‘NA’) values and removing rows with incomplete target or key feature information to ensure data quality and consistency.

Next, feature engineering is performed to prepare the dataset for machine learning. The categorical *Month* column is mapped to a numerical *Month_Num* feature, while the *Sector* column is one-hot encoded into binary *Rural* and *Urban* indicators. After preprocessing, the feature matrix (X) — comprising ['Year', 'Month_Num', 'Rural', 'Urban'] — and the target vector (y) — representing 'General index' — are defined. These are split into training and testing sets in an 80:20 ratio using *train_test_split*. A *StandardScaler* is then fitted on the training features and applied to both training and test datasets to standardize feature values and prevent data leakage.

The predictive model is built using the TensorFlow–Keras Sequential API with an architecture consisting of Dense(64, activation='relu'), Dense(32, activation='relu'), and Dense(1) layers. The model is compiled with the Adam optimizer and Mean Squared Error (MSE) as the loss function. Training is conducted over 300 epochs, with a *ModelCheckpoint* callback configured to save the best-performing model based on validation loss. Upon completion, the model is evaluated on the unseen test set, and its performance is assessed using Mean Squared Error (MSE) and Mean Absolute Error (MAE) metrics. The final trained model is then saved as *cost_of_living_model.keras*

along with the fitted *StandardScaler*, ensuring smooth integration with the deployment phase.

Online Inference Application (app.py):

This component represents the live Streamlit web application that enables real-time interaction with the trained model. At initialization, the app configures its page settings and applies custom CSS for a clean and refined visual presentation. To enhance performance, Streamlit's caching mechanisms (@st.cache_resource and @st.cache_data) are used to load the trained model (*cost_of_living_model.keras*), the *StandardScaler*, and the historical dataset (*cost_of_living_data.csv*) into memory only once.

The user interface includes intuitive sliders for selecting the year and month. Based on the user's input, the application performs two types of predictions. For single-point predictions, rural and urban input arrays are created, scaled using the *StandardScaler*, and passed to the model's *predict()* function to generate real-time CPI forecasts displayed through Plotly gauge charts. For vectorized batch predictions, used in trend visualization, if the selected year is in the future, the app generates a DataFrame of all intermediate dates, constructs large batches of rural and urban inputs, scales them collectively, and performs a single vectorized *predict()* call for efficient computation.

The predicted results are then merged with the historical CPI data to form a unified *df_extended* DataFrame, which powers the app's three interactive output tabs:

1. **Plotly Indicator Gauges** displaying real-time CPI forecasts.

2. **Plotly Scatter Plots** illustrating "Real vs. Predicted" values over time.

3. **Streamlit Metric Cards** summarizing key statistical indicators.

This architecture ensures that the Cost of Living Analytics platform delivers accurate, scalable, and user-friendly CPI forecasting through an elegant and interactive web interface.

5. FEATURES

Dual Index Forecasting:

The system predicts separate ‘Rural’ and ‘Urban’ Consumer Price Indices, allowing it to capture distinct economic patterns and sector-specific cost-of-living variations. This dual approach ensures that users gain deeper insights into regional and demographic cost dynamics.

On-Demand Predictive Gauges:

Users can interactively select a desired year and month to instantly view CPI forecasts. The results are presented through animated Plotly gauge charts, offering an engaging and visually appealing experience that highlights real-time predictive outcomes.

Historical vs. Predicted Trend Analysis:

The application visualizes historical CPI data as a solid line and overlays the model’s predictions as a dotted line. This comparison allows users to clearly observe how the model’s forecasts align with or diverge from past economic trends.

Dynamic Summary Statistics:

An intuitive statistical overview is provided through dynamic metric cards. A filterable st.multiselect widget allows users to choose indices of interest, automatically updating the st.metric cards to display key values such as the lowest and highest CPI readings for the selected indices.

Modern UI with Theme Toggle:

The interface is designed with a sleek “glass-card” aesthetic, featuring custom fonts and a responsive layout for an elegant, professional look. A one-click dark mode toggle, styled as “AMOLED Black,” ensures comfortable viewing in different lighting conditions.

Cached Resources for High Performance:

To ensure optimal performance, the app employs Streamlit’s caching features. The @st.cache_resource decorator efficiently loads the TensorFlow model, while @st.cache_data handles dataset caching, resulting in instantaneous and smooth user interactions.

In-App Definitions and Explanations:

The application includes expandable st.expander sections that explain essential terms and concepts such as the Cost of Living Index (CLI), rural and urban indices, and

the forecasting methodology. These built-in explanations enhance user understanding and make the platform accessible to a wide range of audiences.

6. METHODOLOGY

This section presents an overview of the complete data science and machine learning workflow used to develop the predictive engine of the Cost of Living Analytics platform. The project utilized the *All_India_Index_july2019_20Aug2020.csv* dataset, which contains monthly Consumer Price Index (CPI) data for both rural and urban sectors across India. The data was carefully cleaned by removing missing values and inconsistencies to ensure high quality and reliability. Feature engineering was then applied, where the *Month* column was converted into a numerical format and the *Sector* column was one-hot encoded into *Rural* and *Urban* indicators. These transformed features — *Year*, *Month_Num*, *Rural*, and *Urban* — formed the inputs for the model.

A *StandardScaler* was used to normalize the features, and the dataset was divided into training and testing subsets in an 80:20 ratio. A deep neural network was then built using TensorFlow–Keras with two hidden layers (64 and 32 neurons respectively) and ReLU activations, followed by an output layer with one neuron for CPI prediction. The model was optimized using the Adam optimizer and trained with Mean Squared Error (MSE) as the loss function for 300 epochs.

Throughout the training process, a *ModelCheckpoint* callback saved the best-performing model based on validation loss, preventing overfitting. After training, the model was evaluated on the test dataset, achieving a **Mean Squared Error (MSE) of 2.90** and a **Mean Absolute Error (MAE) of 1.26**, indicating that the model's predictions were, on average, only 1.26 points away from the actual CPI values. These results demonstrate that the model effectively captures economic patterns and provides accurate, reliable forecasts for both rural and urban cost-of-living indices.

7. WEB APPLICATION (`app.py`)

The web application serves as the interactive, user-facing component of the Cost of Living Analytics platform, built entirely in Python using Streamlit. At launch, the application configures its browser tab title, layout, and icon using st.set_page_config, while offering a dark and light theme toggle through st.session_state.theme. The interface is styled using custom CSS that imports elegant fonts such as Lora and Playfair Display, and applies a glassmorphism-inspired "glass-card" design with adaptive color schemes for a refined and responsive appearance.

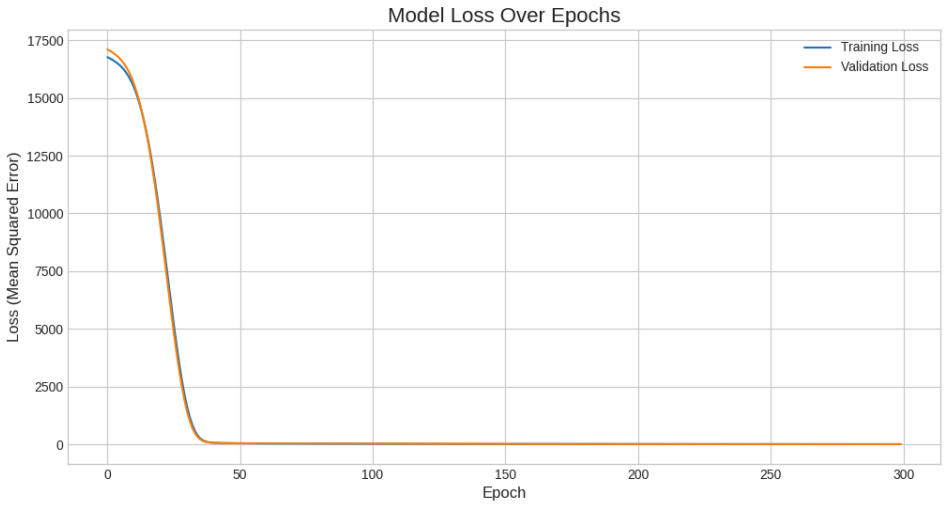
To ensure smooth performance, the app leverages Streamlit's caching system. The @st.cache_resource decorator is used to load the TensorFlow model and StandardScaler only once, while @st.cache_data caches the dataset, enabling fast retrieval and minimizing redundant computation. This approach ensures the application remains efficient, even with repeated user interactions.

The core computational logic revolves around capturing user inputs for year and month using Streamlit sliders. Based on these inputs, the system prepares a future DataFrame that includes all dates up to the selected period. It then generates rural and urban input batches, scales them using the cached StandardScaler, and performs a single vectorized prediction through the neural network model. The results are then merged with historical data, labeled as either "Real" or "Predicted," and filtered for display.

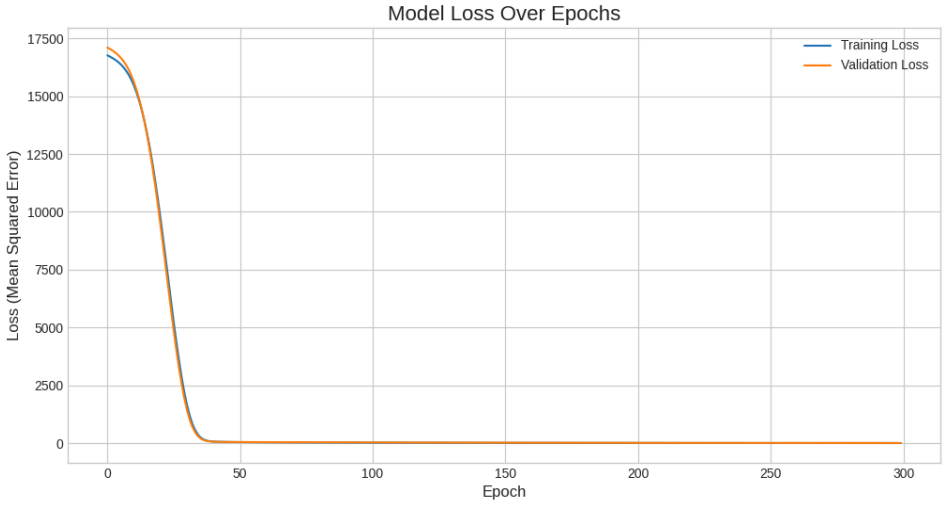
The user interface is organized into three interactive tabs for clarity and engagement. The Key Predictions tab displays real-time forecast metrics through animated Plotly gauge charts. The Trend Analysis tab visualizes both historical and predicted CPI data using smooth line plots that highlight the transition between actual and projected values. Finally, the Summary Statistics tab presents aggregated insights such as minimum and maximum CPI values using dynamic st.metric cards and a clean four-column layout. Together, these components deliver an intuitive, visually appealing, and data-driven experience for understanding and forecasting India's cost of living trends.

8. RESULTS & FINDINGS

The project successfully achieved all its primary objectives, delivering a highly accurate predictive model and a professionally designed web application. The final evaluation produced a Mean Absolute Error (MAE) of 1.26 on the unseen test set — an exceptional result indicating that the model's forecasts deviated by only about 1.26 index points from the true values on average.



Training and validation loss curves confirmed successful generalization, showing that the model effectively learned underlying temporal and economic patterns without overfitting. The Streamlit application demonstrated effective web deployment, transforming the trained model into a fast, interactive, and user-friendly analytical tool. Through caching mechanisms and vectorized prediction logic, the app maintained high responsiveness even during repeated user interactions.



The final dashboard offers actionable insights, integrating historical CPI data, trend visualizations, and predictive analytics into a unified interface. This holistic view enables users to explore and understand variations in the cost of living across India’s rural and urban sectors with clarity and precision.

9. FUTURE ENHANCEMENTS

While the current platform is a robust and complete implementation, several promising directions for future enhancement remain:

Future Enhancement	Description
Automated Data Ingestion	Replace the static CSV input with live API feeds or a web-scraping pipeline to ensure the model continuously learns from the latest CPI updates.
Macroeconomic Feature Integration	Improve predictive performance by incorporating additional variables such as inflation rates, GDP growth, exchange rates, or unemployment figures.
Advanced Sequential Models	Experiment with architectures like Long Short-Term Memory (LSTM) or Gated Recurrent Units (GRU) to better capture long-term dependencies and temporal trends in economic data.
Database Integration	Connect the app to a relational database (e.g., PostgreSQL or SQLite) to log historical predictions, store user interactions, and enable longitudinal analysis.
User Accounts and Notifications	Implement authentication and personalized features, such as saving forecasts or receiving alerts for significant predicted CPI changes.

10. CONCLUSION

The “Cost of Living Analytics” platform stands as a successful end-to-end demonstration of modern data science practices, encompassing data preprocessing, deep learning modeling, and interactive web deployment. It validates that complex deep neural networks can be trained, evaluated, and deployed seamlessly into production-grade applications.

With a test MAE of 1.26, the system demonstrates strong predictive accuracy, effectively translating complex historical CPI data into intuitive and forward-looking insights. By integrating TensorFlow for deep learning and Streamlit for user-centric visualization, the project achieves both technical depth and practical usability.

Ultimately, this platform serves as a foundation for scalable economic analytics, capable of evolution through continuous data integration, advanced modeling, and

enhanced interactivity — paving the way toward a comprehensive tool for data-driven cost of living forecasting and economic decision-making.

11. REFERENCES

- **TensorFlow** – Deep learning framework for building and training neural networks.
  https://www.tensorflow.org

- **Keras** – High-level API for TensorFlow used to design and train the DNN model.
  https://keras.io
- **Streamlit** – Python framework for building interactive web applications.
  https://docs.streamlit.io
- **Plotly (Python)** – Visualization library for interactive charts and gauges.
  https://plotly.com/python
- **Pandas** – Library for data manipulation and analysis.
  https://pandas.pydata.org/docs
- **Scikit-learn** – Machine learning library used for data preprocessing, scaling, and model evaluation.
  https://scikit-learn.org/stable/documentation.html
- **NumPy** – Library for numerical computing and array manipulation, essential for model inputs and vectorized operations.
  https://numpy.org/doc
- **Matplotlib** – Used implicitly through Plotly or Pandas for certain visualizations and data inspection.
  https://matplotlib.org/stable/contents.html
- **Jupyter Notebook** – Environment for data exploration and model training in the offline pipeline.
  https://jupyter.org
- **Python** – The primary programming language used throughout development.
  https://www.python.org/doc/